



SKILLING-ML/DL
II-II CSE, CSE(AI&ML)
AY: 2025-2026

SL No	Week	Session1 (3 Hours)	Detailed Subtopics	Session2 (3 Hours)	Detailed Subtopics
1	Week 5	Artificial Neural Networks (ANN)	Biological neuron inspiration, Mathematical neuron model, Perceptron learning rule, Feedforward neural networks, Activation functions (softmax, Sigmoid, Tanh, ReLU)	ANN Implementation	Loss functions (MSE, Cross-entropy), Forward propagation, ANN architecture design, Binary and multiclass classification

CONTENTS (Week 5 -Session1: Artificial Neural Network)

- 1) Introduction to Artificial Neural Networks
- 2) Motivation for Neural Networks
- 3) Biological Neuron Structure
- 4) Mathematical Model of an Artificial Neuron
- 5) Linear vs Non-Linear Models
- 6) Activation Functions
- 7) Perceptron Model
- 8) Perceptron Learning Rule
- 9) Multi-Layer Feedforward Neural Networks
- 10) Architecture of Neural Networks
- 11) Neural Network Training

Prepared by:

Asha. M

1. What is an Artificial Neural Network?

An **Artificial Neural Network (ANN)** is a computational model inspired by the human brain that is designed to recognize patterns and learn from data.

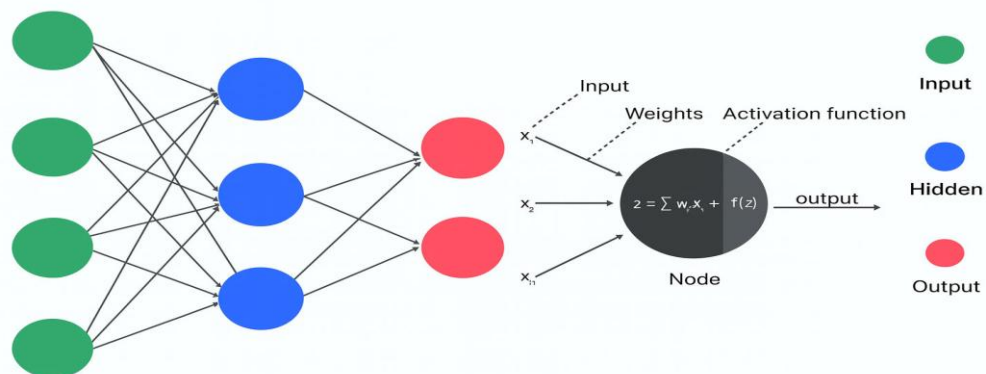
It consists of:

- Neurons (processing units)
- Connections (weights)
- Layers (input, hidden, output)

ANNs are mainly used for:

- Classification
- Regression
- Pattern recognition
- Prediction tasks

Neural Network Architecture



1.1 Why Do We Need ANN?

Traditional algorithms struggle when:

- Data is highly nonlinear
- Feature relationships are complex
- The problem has many variables

ANNs can:

- Learn complex nonlinear mappings
- Automatically adjust weights
- Improve performance with more data

1.2 Key Characteristics of ANN

1. Learns from examples

2. Adjusts internal parameters (weights)
3. Generalizes to unseen data
4. Works well for nonlinear problems

1.3 Simple Real-World Example

Suppose we want to predict:

Will a student pass or fail?

Inputs:

- Study hours
- Attendance percentage

ANN learns:

- How much importance (weight) to give each input
- How to combine them
- How to output pass/fail

Instead of manually writing rules, the network learns automatically from data.

2. Motivation for Neural Networks

2.1 Why Were Neural Networks Developed?

Traditional machine learning models (like linear regression or logistic regression) work well when:

- The relationship between input and output is linear
- The decision boundary is a straight line or simple curve

But many real-world problems are nonlinear and complex.

Neural Networks were developed to handle such complexity.

2.2 Problem with Linear Models

Consider a binary classification problem:

Example:

Classify students as Pass or Fail based on:

- Study hours
- Sleep hours

If the relationship is linear $\rightarrow$ a straight line can separate classes.

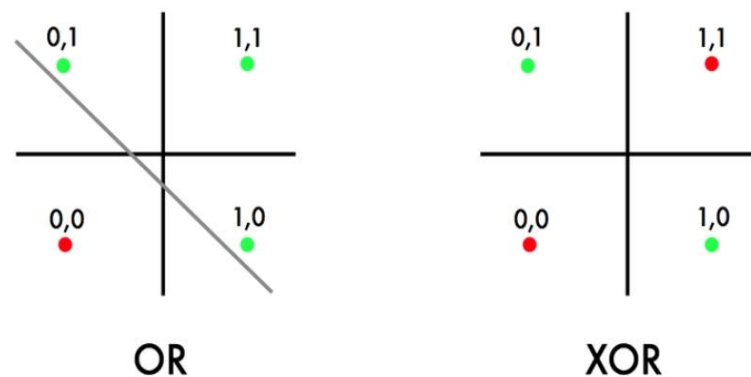
But what if:

- Too little sleep $\rightarrow$ fail
- Too much sleep $\rightarrow$ fail
- Balanced sleep $\rightarrow$ pass

This forms a **nonlinear pattern**.

A straight line cannot solve it.

Classic Example: XOR Problem



This XOR dataset is **not linearly separable**.

A single-layer perceptron fails here.

Neural networks with hidden layers can solve it.

2.3 Real-World Complexity

Many problems involve:

- Multiple features
- Hidden interactions
- Hierarchical patterns
- Nonlinear decision boundaries

Examples:

- Image recognition (pixels combine to form edges → shapes → objects)
- Speech recognition (sound waves → phonemes → words)
- Medical diagnosis (multiple risk factors interacting)

Linear models cannot capture such hierarchical representations.

2.4 What Neural Networks Provide

Neural Networks:

1. Learn nonlinear functions
2. Automatically adjust weights
3. Build hierarchical feature representations
4. Approximate almost any function (Universal Approximation idea)

Key Insight

Traditional ML: You manually engineer features.

Neural Networks: The model automatically learns features from data.

3. Biological Neuron Structure

3.1 Why Study Biological Neurons?

Artificial Neural Networks are inspired by the **human brain**.

Understanding the biological neuron helps students connect intuition to mathematics.

The human brain contains billions of neurons connected through synapses.

3.1.1 Main Parts of a Biological Neuron

A biological neuron has four major components:

a) Dendrites

- Receive signals from other neurons
- Act as input channels

b) Cell Body

- Processes incoming signals
- Performs integration (summation)

c) Axon

- Sends the processed signal forward

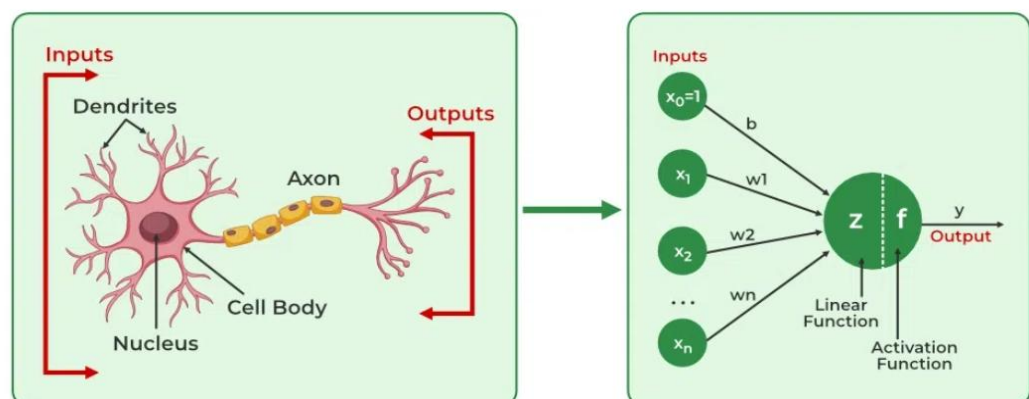
d) Synapse

- Connection between neurons
- Determines signal strength

3.1.2 How Signal Flows in a Biological Neuron

1. Dendrites receive electrical signals
2. Cell body sums the signals
3. If signal exceeds threshold → neuron fires
4. Signal travels through axon
5. Passed to next neuron via synapse

Important concept: A neuron fires only when input exceeds a threshold.



3.2 Mapping Biological Neuron to Artificial Neuron

Biological Component	Artificial Equivalent
Dendrites	Inputs ($x_1, x_2, \dots$)
Synapse	Weights ($w_1, w_2, \dots$)
Cell body	Summation function
Threshold	Bias
Axon	Output
Firing	Activation function

Insight:

Biological neuron: Signal either fires or doesn't fire.

Artificial neuron: Uses mathematical activation function to simulate firing.

Simple Classroom Example

Ask students:

"If 5 friends tell you a movie is good, and 1 says it is bad — how do you decide?"

Each friend's opinion = input

Importance you give each friend = weight

Final decision = output

4. Mathematical Model of an Artificial Neuron

What Can One Neuron Do?

A single neuron can learn one simple pattern.

Mathematically, it creates:

$$w^T x + b = 0$$

Which represents **one decision boundary (a hyperplane)**.

Examples:

- "Larger houses tend to be more expensive."
- "This email looks like spam."
- "Is this a cat? (Yes/No)"

Insight:

A single neuron = One linear pattern detector.

What Can't One Neuron Do?

Real-world problems are rarely governed by a single pattern.

A single neuron cannot capture:

- House price depends on:
 - Square footage
 - Neighborhood
 - Market conditions
- “Image contains both a cat AND a dog”

Because: A single neuron → Only one decision boundary.

Complex relationships require multiple interacting patterns.

Why One Pattern Is Not Enough

Consider house price prediction:

- Pattern 1: Bigger houses cost more
- Pattern 2: Good neighborhoods increase price
- Pattern 3: Older houses depreciate

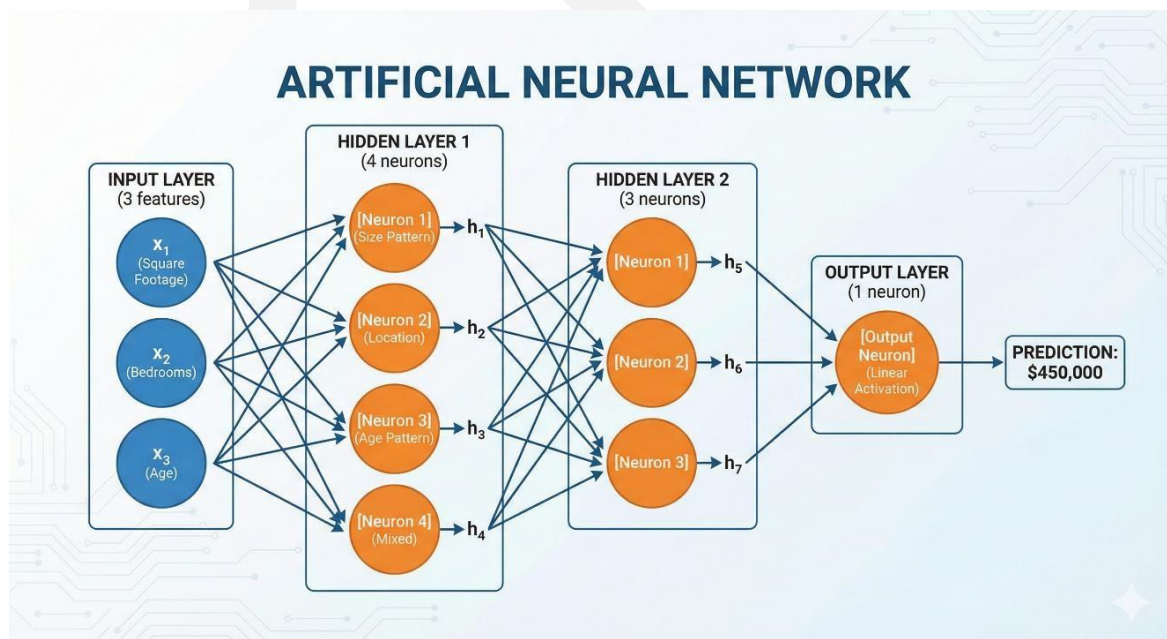
A single neuron can only learn **one** of these at a time.

To learn them simultaneously: We need multiple neurons working in parallel.

The Solution: Multiple Neurons in Parallel

Instead of: Input → One Neuron → Output

We build: Input → Multiple Neurons → Multiple Outputs



Key Insight:

Each neuron:

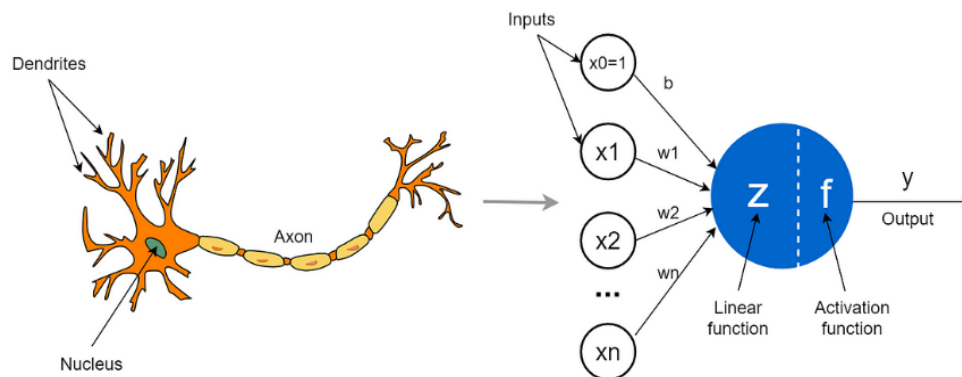
- Receives the same input
- Has different weights
- Has different bias
- Learns a different pattern

Even though inputs are identical, outputs differ because parameters differ.

4.1 Structure of an Artificial Neuron

An artificial neuron consists of:

- Inputs: $x_1, x_2, x_3, \dots, x_n$
- Weights: $w_1, w_2, w_3, \dots, w_n$
- Bias: b
- Activation Function: $f(\cdot)$
- Output: y



Step 1: Weighted Sum (Linear Combination)

Each input is multiplied by its corresponding weight.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

This is also written in vector form:

$$z = w^T x + b$$

Where:

- w^T = transpose of weight vector
- x = input vector

a) Why Weights?

Weights determine:

- Importance of each feature

- Direction of influence (positive or negative)

Example:

If weight is large → strong influence

If weight is negative → inverse relationship

For 2 inputs:

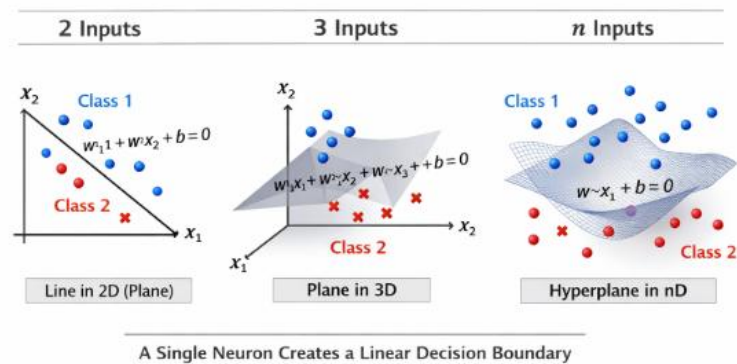
$$w_1x_1 + w_2x_2 + b = 0$$

This represents a **line** in 2D space.

For 3 inputs → a plane.

For n inputs → hyperplane.

So a single neuron creates a **linear decision boundary**.



Weights control:

- Feature importance
- Direction of boundary
- Model behavior

b) Why Bias?

Bias allows shifting the decision boundary.

Without bias:

The decision boundary always passes through origin.

With bias:

The model gains flexibility.

Then decision boundary always passes through origin.

Bias shifts the boundary.

Analogy:

Weights decide slope.

Bias decides intercept.

Step 2: Activation Function

After computing z , we apply a nonlinear function:

$$y = f(z)$$

This introduces nonlinearity into the model.

Without activation: The neuron behaves like linear regression.

Complete Neuron Equation

$$y = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$$

This is the fundamental equation of neural networks.

Numerical Example: Suppose

Inputs:

- Study hours (x_1) = 4
- Attendance (x_2) = 80

Weights:

- $w_1 = 0.6$
- $w_2 = 0.02$

Bias:

- $b = -1$

Step 1: Weighted sum

$$z = (0.6 \times 4) + (0.02 \times 80) - 1$$

$$z = 2.4 + 1.6 - 1 = 3$$

Step 2: Apply activation (for now assume step function)

If $z > 0 \rightarrow$ Pass

If $z \leq 0 \rightarrow$ Fail

Since $3 > 0 \rightarrow$ Output = Pass

Insight: Artificial neuron performs:

1. Multiply
2. Add
3. Apply function

That's it.

The power comes when we connect many such neurons.

5. Linear vs Non-Linear Models

5.1 What is a Linear Model?

A model is linear if the output is a linear combination of inputs.

$$y = w_1x_1 + w_2x_2 + b$$

Characteristics:

- Decision boundary is a straight line (2D)
- A plane (3D)
- A hyperplane (nD)
- Cannot capture complex patterns

Examples:

- Linear Regression
- Logistic Regression
- Single-layer Perceptron

5.2 Linear Decision Boundary

For classification:

$$w_1x_1 + w_2x_2 + b = 0$$

This separates data into two halves.

If:

$$w^T x + b > 0 \Rightarrow \text{Class1}$$

$$w^T x + b < 0 \Rightarrow \text{Class2}$$

Only works if data is **linearly separable**.

5.3 What is a Non-Linear Model?

A model is **nonlinear** if:

- It uses nonlinear transformations
- It can create curved or complex decision boundaries

Mathematically:

$$y = f(w^T x + b)$$

Where f is a nonlinear activation function.

5.4 Why Linear Models Fail

Consider XOR:

x1	x2	Output
0	0	0
0	1	1
1	0	1
1	1	0

No straight line can separate this.

Linear model fails.

5.5 How Neural Networks Become Nonlinear

Nonlinearity comes from:

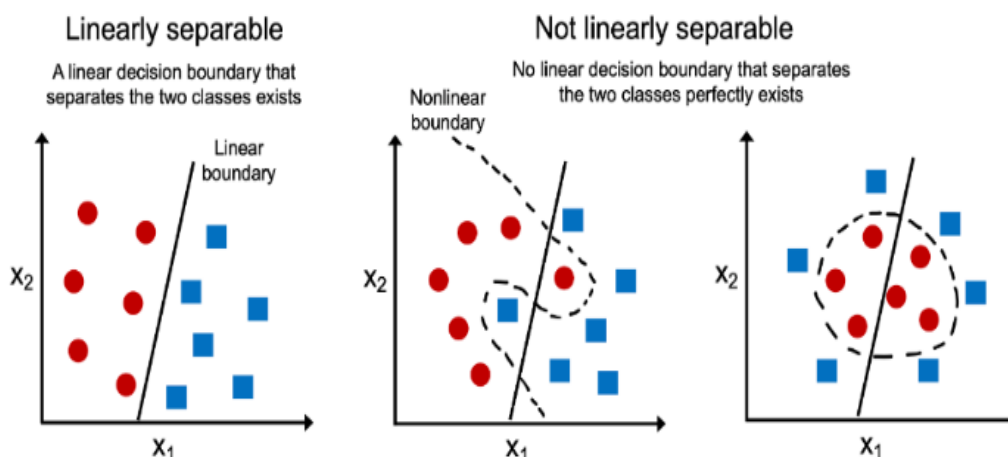
- Activation functions
- Hidden layers

When multiple nonlinear neurons are stacked:

The network can create:

- Curved boundaries
- Complex shapes
- Arbitrary functions

This is why multilayer neural networks solve XOR.



6. Activation Functions

6.1 Why Do We Need Activation Functions?

Without activation:

$$y = w^T x + b$$

Even if we stack many layers:

$$y = W_3(W_2(W_1x))$$

It is still linear.

Stacking linear layers gives another linear function.

So neural networks would behave like simple linear models.

Activation functions introduce **non-linearity**, which allows:

- Learning complex patterns

- Solving XOR
- Creating curved decision boundaries

6.2 What is an Activation Function?

It is a function applied to the weighted sum:

$$y = f(w^T x + b)$$

It decides whether a neuron should “fire”.

Activation functions serve two main purposes:

1. **Introduce Non-linearity:** Without non-linearity, a neural network would behave like a linear model, no matter how deep it is. Activation functions allow the network to learn and represent complex, non-linear mappings between inputs and outputs.
2. **Control Neuron Activation:** Activation functions control the firing behavior of neurons. Depending on the activation function’s output, a neuron might become activated (output a non-zero value) or remain inactive (output zero).

Typically, same activation function is applied to all the hidden layers, while the output layer uses a different activation function, based on the type of prediction model aims to make. How to choose an activation function is explained later in this article.

6.3 Types of Activation Functions

a) Step Function (Perceptron Activation)

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Characteristics:

- Binary output (0 or 1)
- Not differentiable
- Cannot use gradient-based learning easily

b) Sigmoid Function (Binary Class)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Range:

0 to 1

Properties:

- Smooth and differentiable
- Used for binary classification
- Output interpreted as probability

Problem:

Vanishing gradient for large positive/negative values.

c) Tanh Function

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Range:

-1 to 1

Properties:

- Zero-centered
- Stronger gradients than sigmoid
- Still suffers from vanishing gradient

d) ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z)$$

Range:

0 to ∞

Advantages:

- Computationally efficient
- Reduces vanishing gradient problem
- Sparse activation

Problem:

Dying ReLU (neurons stop activating)

e) Softmax (Multi-Class)

Used in multi-class classification.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Where:

- k = number of classes

Key Properties:

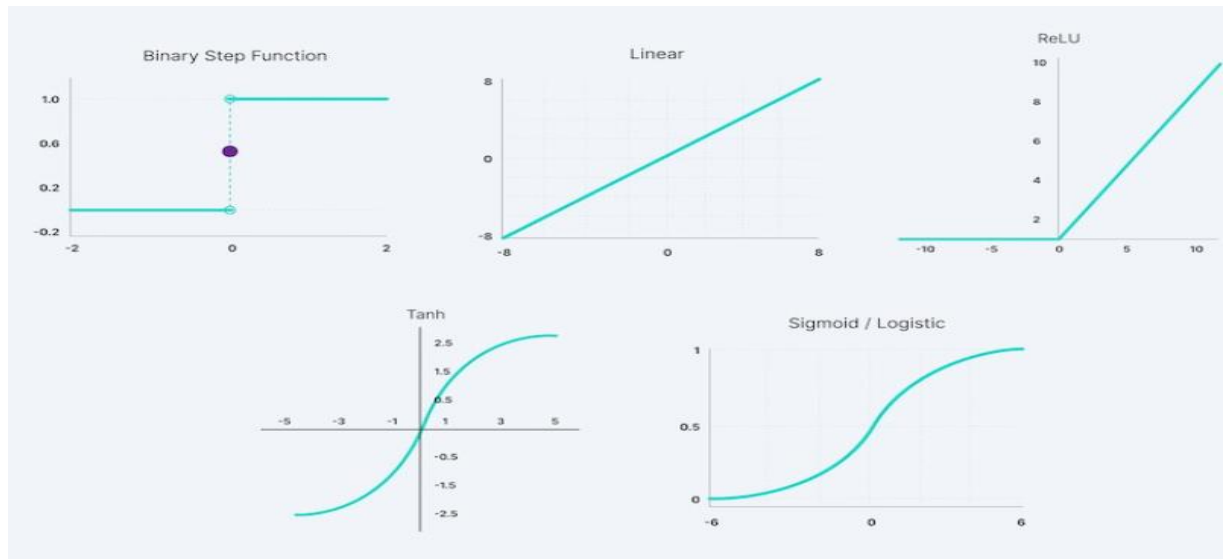
- Outputs probabilities
- Sum of outputs = 1
- Used in output layer for multi-class classification

Example:

If output is [2.1, 1.3, 0.5] $\rightarrow$ Softmax converts to:

[0.62, 0.25, 0.13]

Highest probability = predicted class.



Used in layer	Activation Function	Details	Pros	Cons
Hidden / Output	Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$ Output range: [0,1]	- Smooth activation that outputs values between 0 and 1, making it suitable for binary classification - Historically popular	- Vanishing gradient problem due to saturated neurons - Output not zero-centered as sigmoid outputs are always positive - exp() operation is computationally intensive
Hidden	Tanh (Hyperbolic tangent)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Output range: [-1,1]	- Zero centered outputs that help networks train faster	- Suffers with vanishing gradient problem when saturated
Hidden	ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$ Output range: [0, ∞)	- Does not saturate: avoids vanishing gradient issues for positive inputs - Computationally efficient since only certain number of neurons are activated at the same time - Much faster convergence compared to sigmoid/tanh	- Output not zero-centered - Prone to a "dying ReLU" problem, where neurons can get stuck during training and never activate again, leading to a dead neuron that doesn't update its weights.

© AIML.com Research

Used in layer	Activation Function	Details	Pros	Cons
Hidden	Leaky ReLU	$f(x) = \max(0.01x, x)$ Output range: $(-\infty, \infty)$	- All benefits of ReLU - Addresses the "dying ReLU" problem by allowing a small gradient for negative inputs. Helps with training deeper networks	- Not as standardized as ReLU, and the slope of the leaky part is typically a hyperparameter (α) that needs tuning. This is also referred to as Parametric ReLU $f(x) = \max(\alpha x, x)$
Hidden	ELU (Exponential Linear Unit)	where $\alpha > 0$: $f(x) = \begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$ α is commonly chosen as 1 Output range: $(-\alpha, \infty)$	- All benefits of ReLU - Zero centered outputs that help networks train faster - ELUs saturate to a negative value when the argument gets smaller becoming more robust to noise	- Computationally more expensive due to exponential operation
Output	Softmax	For a given class i , probability $P(y_i)$ is: $P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$ where, z_i is the raw score (logit) for class i , and N is the no. of classes Output range: $(0,1)$	- Used in the output layer of multi-class classification problems, converting model outputs into probability distributions.	- Not suitable for multi-label classification, as it enforces that only one class can be predicted. © AIML.com Research

6.4 Comparison Summary

Activation	Output Range	Differentiable	Functional Role	Main Issue
Step	{0,1}	No	Early perceptron	No gradient learning
Sigmoid	(0,1)	Yes	Binary output layer	Vanishing gradient
Tanh	(-1,1)	Yes	Hidden layers	Vanishing gradient
ReLU	$[0, \infty)$	Yes (except 0)	Hidden layers	Dying neurons
Softmax	(0,1), sum=1	Yes	Multi-class output layer	Sensitive to large values

Which activation function to choose from?

Activation function for hidden layers:

In practice, the choice of activation functions is as follows in order of priority:

- ReLU is the top choice as it is simpler, faster, much lower run time, better convergence performance and do not suffer from vanishing gradient issues
- tanh

Activation function for output layers:

- Sigmoid for binary classification.
- Softmax for multi-class classification
- Identity / linear function for regression problems

Insight: Activation function is what gives neural networks power.

Without it → network = linear model.

With it → network = universal approximator.

Hidden layers → ReLU (mostly)

Binary classification output → Sigmoid

Multi-class classification output → Softmax

7. Perceptron Model

7.1 Introduction

The Perceptron is the first artificial neural network model proposed by Frank Rosenblatt in 1958.

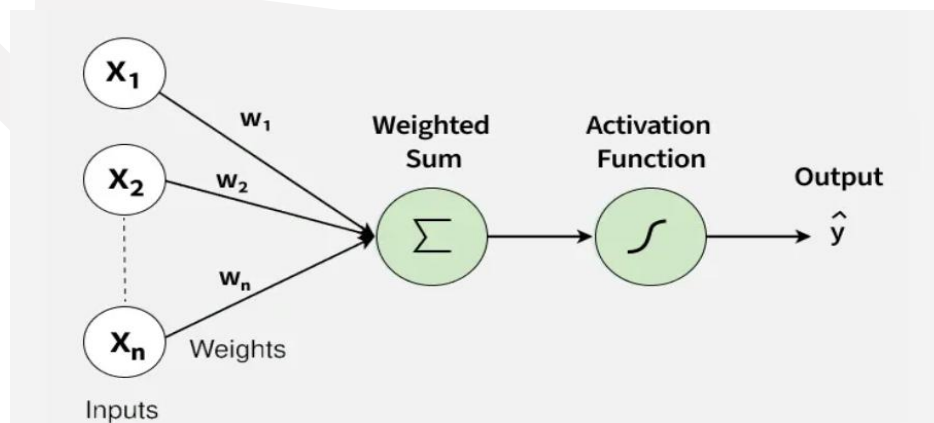
It is a single-layer binary classification model. It is the foundation of modern neural networks.

7.2 What is a Perceptron?

A perceptron is:

- A single artificial neuron
- With weights and bias
- Using a step activation function
- Designed for binary classification

It makes decisions by creating a linear decision boundary.



7.3 Mathematical Representation

Step 1: Linear Combination

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Step 2: Apply Step Activation

$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

This produces a binary output.

7.4 Decision Boundary

The perceptron decision boundary is:

$$w^T x + b = 0$$

This represents:

- A line (2D)
- A plane (3D)
- A hyperplane (nD)

Hence, perceptron is a **linear classifier**.

7.5 Geometric Interpretation

The weight vector w :

- Is perpendicular (normal) to the decision boundary
- Controls orientation

Bias b :

- Shifts the boundary

So:

Weights $\rightarrow$ direction

Bias $\rightarrow$ position

7.6 Assumptions

Perceptron assumes:

- Data is linearly separable
- Binary output
- Deterministic decision

7.7 Limitations

Perceptron:

- Cannot solve non-linear problems
- Fails on XOR

- Uses non-differentiable activation

This limitation led to development of: multi-layer neural networks

8. Perceptron Learning Rule

8.1 How does a perceptron learn?

Learning means:

- Adjusting weights
- Reducing classification errors
- Improving decision boundary

The perceptron updates weights **only when it makes a mistake**.

a) Prediction Step

First compute:

$$z = w^T x + b$$

Apply step function:

$$\hat{y} = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Where:

- $\hat{y}$ = predicted output
- y = actual output

b) Error Calculation

$$\text{Error} = y - \hat{y}$$

If:

- Prediction correct $\rightarrow$ Error = 0 $\rightarrow$ No update
- Prediction wrong $\rightarrow$ Error $\neq$ 0 $\rightarrow$ Update weights

c) Weight Update Rule

$$w = w + \eta(y - \hat{y})x$$

$$b = b + \eta(y - \hat{y})$$

Where:

- η = learning rate
- x = input vector
- $y - \hat{y}$ = error

Weight Update Mechanism (Geometric View)

When prediction is wrong:

$$w = w + \eta(y - \hat{y})x$$

If:

- True class = 1 but predicted = 0
→ Increase weights in direction of input

If:

- True class = 0 but predicted = 1
→ Decrease weights

This shifts the decision boundary toward correct classification.

8.2 Role of Learning Rate (η)

The learning rate (η) is a hyperparameter that controls:

- How much the weights change during each update
- The step size toward minimizing error

If η is:

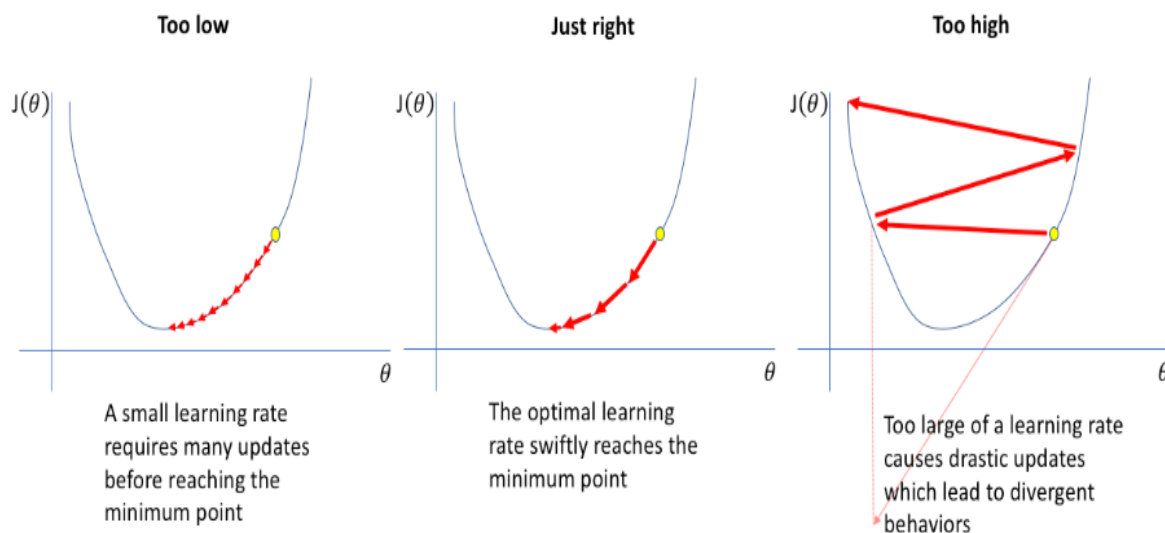
Too Small

- Learning is very slow
- Convergence takes many epochs
- Training time increases

Too Large

- Updates overshoot
- Model may oscillate
- May never converge

So η must be chosen carefully.



Imagine you are walking down a hill:

Small steps → slow but safe

Very large jumps → may miss the minimum

Learning rate = step size.

8.3 Training Process (Algorithm)

For each training example:

1. Compute output
2. Compare with actual label
3. Update weights if incorrect
4. Repeat for multiple epochs

Training stops when:

- All points are correctly classified
OR
- Maximum epochs reached

8.4 Limitation of Learning Rule

- Only works for linear separability
- Cannot minimize differentiable loss
- No probability output

This limitation leads to: Multilayer networks, Gradient descent, Backpropagation

9. Multi-Layer Feedforward Neural Networks

9.1 Why Multi-Layer Networks?

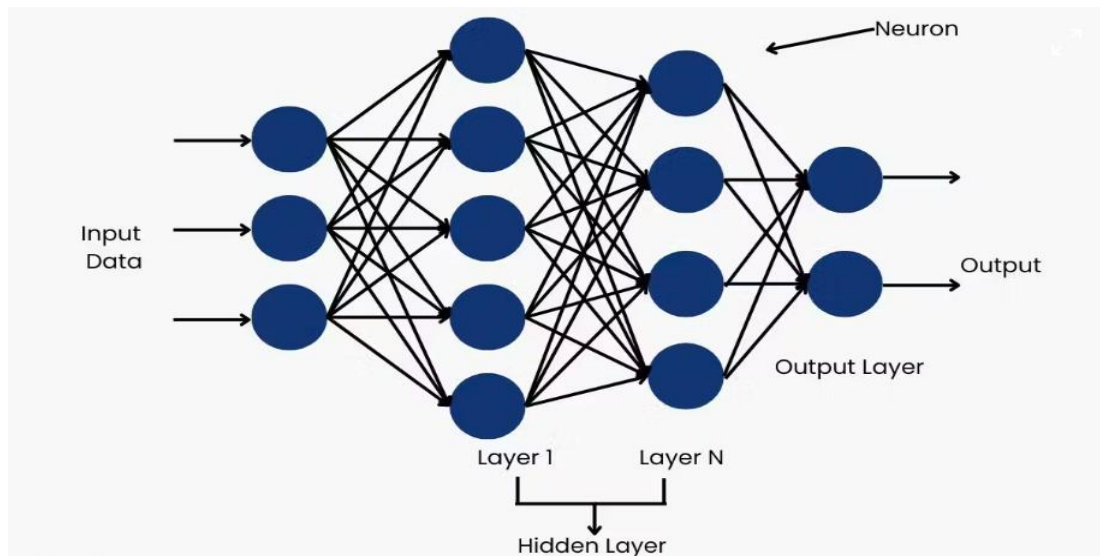
We saw:

- Perceptron → only linear boundary
- XOR → not linearly separable

To solve nonlinear problems, we add:

Hidden layers

This leads to Multi-Layer Neural Networks.



9.2 What is a Multi-Layer Feedforward Network?

A Multi-Layer Feedforward Neural Network is:

- A network with more than one layer of neurons
- Data flows in one direction (input → output)
- No cycles or feedback connections

“Feedforward” means signals move only forward.

9.3 Basic Structure

It consists of:

- 1) Input Layer
- 2) One or more Hidden Layers
- 3) Output Layer

Each layer contains multiple neurons.

9.4 Mathematical Representation

For one hidden layer:

Hidden Layer:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = f(z^{(1)})$$

Output Layer:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$y = f(z^{(2)})$$

Now we have **two linear transformations + nonlinear activations**.

This creates nonlinear decision boundaries.

9.5 Why Hidden Layers Matter

Each hidden neuron:

- Learns intermediate features
- Transforms input space

9.6 Key Advantage

Multi-layer networks can:

- Learn nonlinear mappings
- Model complex patterns
- Approximate almost any continuous function

10. Architecture of Neural Networks

(Input Layer, Hidden Layer, Output Layer)

10.1 What is Network Architecture?

Neural network architecture defines:

- How many layers the network has
- How many neurons are in each layer
- How layers are connected

It determines the model's capacity and learning ability.

a) Input Layer

- Receives raw input features
- Does not perform computation

If dataset has:

- 2 features $\rightarrow$ 2 input neurons
- 10 features $\rightarrow$ 10 input neurons

Example:

Student prediction problem:

- Study hours
- Attendance

Input layer = 2 neurons

Important: Input layer simply passes values forward.

b) Hidden Layer

- Performs transformations

- Extracts patterns
- Learns intermediate representations

Each hidden neuron computes:

$$z = Wx + b$$

$$a = f(z)$$

Hidden layers introduce:

- Nonlinearity
- Feature learning
- Complex decision boundaries

More hidden layers → deeper network.

c) Output Layer

- Produces final prediction

Number of neurons depends on problem:

Binary classification:

- 1 neuron (Sigmoid)

Multi-class classification:

- k neurons (Softmax)

Regression:

- 1 neuron (Linear activation)

10.2 Fully Connected Architecture

In basic feedforward networks:

- Every neuron in one layer connects to every neuron in next layer
- Each connection has a weight

This is called a Dense Layer.

10.3 Important Design Considerations

When designing architecture:

- How many hidden layers?
- How many neurons per layer?
- Which activation function?

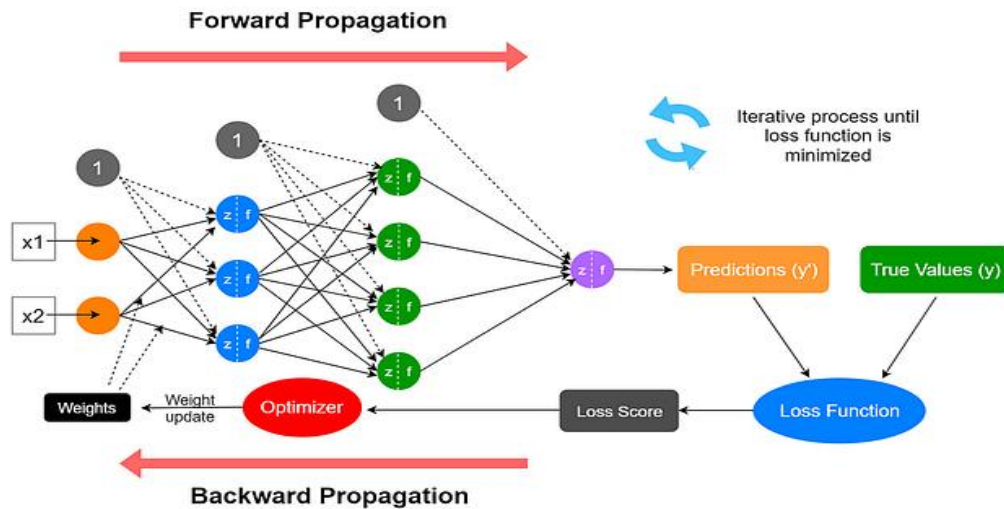
These are hyperparameters.

11. Neural Network Training

Neural network training consists of **three main steps**:

- Forward Propagation** – Pass input through the network to generate predictions.
- Loss Computation** – Calculate the error between predicted output and actual output.
- Backward Propagation** – Update weights by propagating the error backward to minimize the loss.

These three steps repeat iteratively during training.



Now we move into the core computation process of neural networks.

11.1 Forward Propagation

Forward propagation is how input data moves through the network to produce an output.

Forward propagation is the process of:

- Passing input data layer by layer
- Computing weighted sums
- Applying activation functions
- Producing final prediction

Flow direction: Input → Hidden Layer(s) → Output

a) Step-by-Step Mathematical Flow

(Example: 1 Hidden Layer Network)

Step 1: Input to Hidden Layer

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = f(z^{(1)})$$

Where:

- x = input vector
- $W^{(1)}$ = weights of hidden layer
- $b^{(1)}$ = bias
- f = activation function

Step 2: Hidden to Output Layer

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$\hat{y} = g(z^{(2)})$$

Where:

- g = output activation (Sigmoid / Softmax / Linear)

For multiple neurons:

$$Z^{(l)} = W^{(l)}A^{(l-1)} + b^{(l)}$$

$$A^{(l)} = f(Z^{(l)})$$

This is the generalized forward propagation formula.

Why Forward Propagation is Important

- Produces predictions
- Computes values needed for loss
- Stores intermediate values for backpropagation

It is the first phase of training.

Insight:

Forward propagation = Pure computation.

No learning happens here.

Learning happens when we calculate error and update weights.

11.2 What is a Loss Function?

A loss function measures how far the predicted output ($\hat{y}$) is from the true output (y).

It converts prediction error into a single numerical value.

$$L = f(y, \hat{y})$$

If loss is:

- Small $\rightarrow$ Model is performing well
- Large $\rightarrow$ Model is performing poorly

11.2.1 Why Do We Need Loss?

Without loss:

- We don't know how wrong the model is
- We cannot improve weights
- Learning cannot happen

Loss provides the **objective** the model tries to minimize.

11.2.2 Types of Loss Functions

Loss function depends on the problem type.

a) Regression Problems

Mean Squared Error (MSE)

$$L = \frac{1}{n} \sum (y - \hat{y})^2$$

Used when:

- Output is continuous
- Example: House price prediction

Why square?

- Makes errors positive
- Penalizes large errors more

b) Binary Classification

Binary Cross-Entropy (Log Loss)

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Used when:

- Output is 0 or 1
- Final activation is Sigmoid

Example:

Spam detection

Disease prediction

c) Multi-Class Classification

Categorical Cross-Entropy

$$L = -\sum y_i \log(\hat{y}_i)$$

Used when:

- Multiple classes
- Final activation is Softmax

Example:

Digit classification (0–9)

Cross-entropy:

- Punishes confident wrong predictions heavily
- Rewards confident correct predictions

If model predicts:

- 0.99 for correct class → very small loss
- 0.99 for wrong class → very large loss

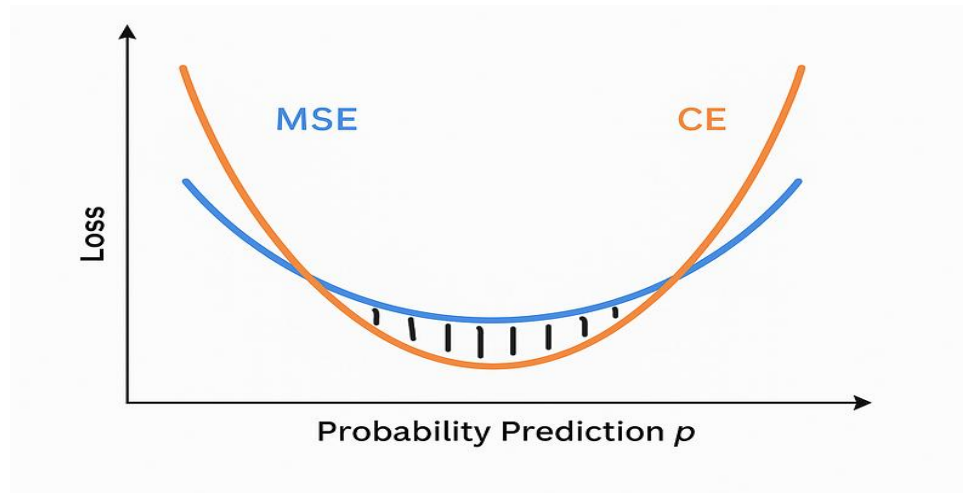
This makes learning stable and effective.

For MSE:

Loss curve is a parabola (smooth).

For Cross-Entropy:

Loss increases sharply for wrong confident predictions.



Insight:

Loss Function = “Error Measuring Tool”

Different problems → Different loss functions

We always try to minimize loss.

11.3 Backpropagation

Forward Propagation → makes prediction

Loss Function → measures error

Backpropagation → corrects the model

Backpropagation is the algorithm used to:

- Compute how much each weight contributed to the error
- Adjust weights to reduce the loss

It uses:

- Chain Rule from calculus
- Gradient Descent optimization

11.3.1 Why Do We Need It?

Imagine:

Prediction = 0.9

Actual = 1

Loss is small.

But how do we know:

- Which weight caused the error?
- How much should each weight change?

Backpropagation answers this.

11.3.2 Core Idea

Backpropagation computes:

$$\frac{\partial L}{\partial W}$$

Meaning:

“How does loss change if this weight changes?”

This is called the **gradient**.

11.3.3 Direction of Flow

Forward pass:

Input → Hidden → Output

Backward pass:

Output → Hidden → Input

Error moves backward through the network.

11.3.4 Weight Update Rule

After computing gradient:

$$W = W - \eta \frac{\partial L}{\partial W}$$

Where:

- η = learning rate
- Gradient tells direction
- Learning rate controls step size

Insight: Think of climbing down a hill.

Loss = height of hill

Gradient = slope direction

Learning rate = step size

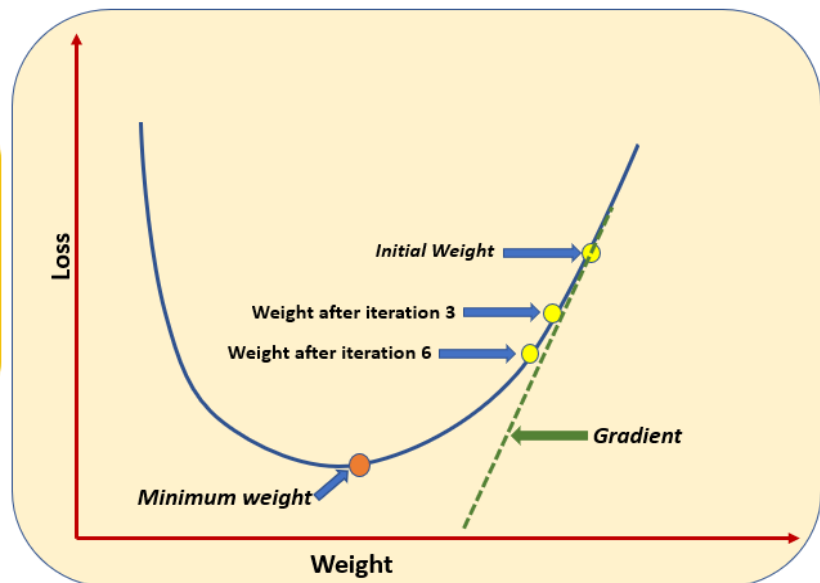
Goal:

Reach the minimum loss.

Update Rule

$$w_{new} = w_{old} - \alpha \left(\frac{\partial L}{\partial w} \right)_{w=w_{old}}$$

L is the Loss function, w is the weight and α is the learning rate



11.3.5 What Backpropagation Actually Does Internally

For each layer:

1. Compute output error
2. Compute gradient
3. Propagate error to previous layer
4. Update weights

Repeat for all layers.

11.3.6 Why It's Powerful

Without backpropagation:

- Deep networks cannot be trained
- Multi-layer learning is impossible

Backpropagation made modern deep learning possible.

Q&A

1) Why is bias necessary in a neuron?

Answer: Without bias, the decision boundary must pass through the origin.

Bias allows shifting the hyperplane, increasing representational flexibility.

2) A perceptron perfectly classifies training data but fails on XOR. Why?

Answer: Because XOR is not linearly separable.

A single perceptron creates only a linear decision boundary, while XOR requires a non-linear boundary.

3) If learning rate is extremely high, what happens during training?

Answer:

The model may:

- Overshoot minima
- Oscillate
- Diverge

Training becomes unstable.

4) Can a single neuron approximate any function?

Answer:

No.

A single neuron produces only a linear decision boundary.

Non-linear complex functions require multiple neurons/layers.

5) Why do we square error in MSE?

Answer:

- Makes error positive
- Penalizes large errors more
- Creates smooth differentiable surface

6) If all weights initialize to zero, what happens?

Answer:

All neurons in the same layer learn identical features.

Symmetry is not broken → network fails to learn diverse representations.

7) Why is cross-entropy preferred over MSE for classification?

Answer:

Cross-entropy:

- Aligns with probability theory
- Produces stronger gradients
- Penalizes confident wrong predictions more

Leads to faster convergence.

8) What does backpropagation actually compute?

Answer:

It computes gradients of loss with respect to every weight:

$$\frac{\partial L}{\partial W}$$

Using the chain rule efficiently.

9) Why is non-linearity required in hidden layers but not always in output?

Answer:

Hidden layers require non-linearity for expressive power.

Output activation depends on task:

- Regression → Linear
- Binary → Sigmoid
- Multi-class → Softmax

10) What happens if we remove hidden layers completely?

Answer: The network reduces to logistic regression (for classification) or linear regression (for regression).

11) Why does deep learning require differentiable activation functions?

Answer:

Backpropagation requires gradients.

Non-differentiable functions prevent gradient computation.

12) Why can a multi-layer network solve XOR but perceptron cannot?

Answer: Because hidden layers create intermediate feature transformations.

They map input into a space where data becomes linearly separable.

13) Scenario: Model Predicts Only One Class

You train a binary classifier using sigmoid activation.

After training, the model predicts **only class 1** for all inputs.

What could be the possible reasons?

Answer:

Possible causes:

- Severe class imbalance
- Learning rate too high or too low
- Poor weight initialization
- Bias dominating weights
- Vanishing gradients (if deep network)

Conceptually:

The decision boundary may have shifted far to one side, making all inputs fall into the same half-space.

14) Scenario: Training Loss Decreases, But Accuracy Doesn't Improve

You observe:

Loss ↓ steadily

Accuracy $\approx$ constant

Why can this happen?

Answer:

Because:

- Loss measures confidence, accuracy does not
- Predictions may become more confident but still wrong
- In classification, probabilities might improve without crossing 0.5 threshold

This shows:

Loss is continuous, accuracy is discrete.

15) Scenario: Extremely Small Learning Rate

You train a perceptron with learning rate = 0.0000001.

What happens during training?

Answer:

- Weight updates become extremely tiny
- Convergence becomes very slow

Gradient descent steps are too small to move toward minimum efficiently.